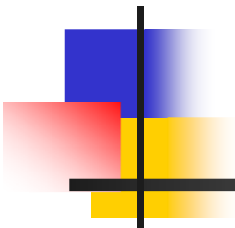


第4-5章

数据挖掘的数据仓库与 OLAP技术



(Online Analytical Processing,
联机分析处理)

第4-5章: 数据挖掘的数据仓库与OLAP技术

- **什么是数据仓库?**
- **多维数据模型**
- **数据仓库结构**
- **数据仓库实现**
- **数据立方体的进一步发展**
- **从数据仓库到数据挖掘**

数据仓库诞生的原因

- **历史数据积存**：历史数据使用频率过低，堆积在业务数据库中，会导致查询性能下降
- **企业数据分析需要**：各个部门自己建立独立的数据抽取系统，导致数据不一致，资源浪费严重，数据库权限也会存在风险



数据仓库基本概念？

数据仓库（Data Warehouse）顾名思义，是一个很大的数据存储集合，出于企业的分析性报告和决策支持目的而创建，对多样的业务数据进行筛选与整合。它为企业提供一定的BI（商业智能）能力，指导业务流程改进、监视时间、成本、质量以及控制。

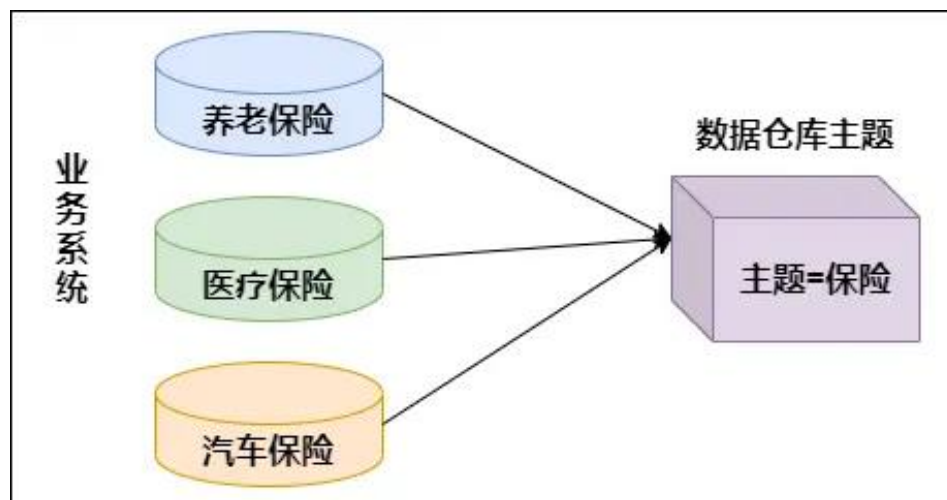
数据仓库将各个异构的数据源数据库的数据给统一管理起来，并且完成了质量较差的数据的剔除、格式转换，最终按照一种合理的建模方式来完成源数据组织形式的转变，以更好的支持到前端的可视化分析。数据仓库的输入方式是各种各样的数据源，最终的输出用于企业的数据分析、数据挖掘、数据报表等方向。

数据仓库定义

- 有不同的方法定义, 但不是严格的.
 - 是一个决策支持数据库, 它与组织机构的操作数据库分别维护
 - 数据仓库系统允许将各种应用系统集成在一起, 为统一的历史数据分析提供坚实的平台, 支持信息处理
- W. H. Inmon(数据仓库之父)的定义: 数据仓库是
 - 面向主题的(subject- oriented),
 - 集成的(integrated),
 - 时变的(time-variant), 和
 - 非易失的(nonvolatile) 数据集, 支持管理决策过程

数据仓库—面向主题的

- 不同于传统数据库对应于某一个或多个项目，数据仓库根据使用者实际需求，将不同数据源的数据在一个较高的抽象层次上做整合，所有数据都**围绕某一主题**来组织。
- 关注决策制定者的数据建模与分析, 而不是日常的操作和事务处理.
- 数据仓库排除对于决策过程无用的数据, 提供特定主题的简明视图.



数据仓库— 集成的

- 数据仓库中存储的数据是来源于多个数据源的集成，原始数据来自不同的数据源，存储方式各不相同。要整合成为最终的数据集合，需要从数据源经过一系列抽取、清洗、转换的过程。
- 通过将多个异种的数据源集成在一起, 而构造
 - 比如，关系数据库, 一般文件, 联机事务记录
- 使用数据清理和数据集成技术.
 - 确保命名约定, 编码结构, 属性度量等的一致性
 - 例如, 饭店价格: 货币种类, 税, 是否含早餐, 等.
 - 当数据装入数据仓库时, 数据将被转换.

数据仓库— 时变的

- **数据仓库的时间跨度显著地比操作数据库长.**
 - **操作数据库数据: 当前值数据.**
 - **数据仓库数据: 从历史的角度提供数据 (例如, 过去 5-10年)**
- **数据仓库中的每个键结构**
 - **显式或隐式地包含时间元素,**
 - **但是, 操作数据的键可能包含, 也可能不包含“时间元素”.**

数据仓库—非易失的

- 从操作环境转换过来的数据物理地分离存放.
- 数据的更新不在数据仓库环境中出现.
 - 不需要事务处理, 恢复, 和并发控制机制
 - 只需要两种数据存取操作:
 - 数据的初始化装入 和 数据访问.

数据仓库VS.操作数据库

数据库：数据库是面向交易的处理系统（业务系统），它是针对具体业务在数据库联机的日常操作，通常对记录进行查询、修改。用户较为关心操作的响应时间、数据的安全性、完整性和并发支持的用户数等问题。传统的数据库系统作为数据管理的主要手段，主要用于操作型处理，也被称为联机事务处理 OLTP（On-Line Transaction Processing）。

数据仓库：数据仓库一般针对某些主题的历史数据进行分析，支持管理决策，又被称为联机分析处理 OLAP（On-Line Analytical Processing）。

数据仓库VS.操作数据库-区别

- 1、数据库是面向事务的设计，数据仓库是面向主题设计的。
- 2、数据库一般存储业务数据，数据仓库存储的一般是历史数据。
- 3、数据库设计是尽量避免冗余，一般针对某一业务应用进行设计，比如一张简单的User表，记录用户名、密码等简单数据即可，符合业务应用，但是不符合分析。数据仓库在设计是有意引入冗余，依照分析需求，分析维度、分析指标进行设计。
- 4、数据库是为**捕获数据**而设计，数据仓库是为**分析数据**而设计。
数据仓库的出现，并不是要取代数据库

数据仓库VS.操作数据库-示例

以银行业务为例：

- 数据库是事务系统的数据平台，客户在银行做的每笔交易都会写入数据库，被记录下来，这里，可以简单地理解为用数据库记账。
- 而数据仓库是分析系统的数据平台，它从事务系统获取数据，并做汇总、加工，为决策者提供决策的依据。
 - 比如，某银行某分行一个月发生多少交易，该分行当前存款余额是多少。如果存款又多，消费交易又多，那么该地区就有必要设立ATM了。

数据仓库VS.操作数据库-示例

显然，银行的交易量是巨大的，通常以百万甚至千万次来计算。

事务系统是实时的，这就要求**时效性**

- 客户存一笔钱需要几十秒是无法忍受的，这就要求数据库只能存储很短一段时间的数据。

而分析系统是事后的，它要提供关注时间段内所有有效数据。这些数据是海量的，汇总计算起来也要慢一些，但是，只要能够提供有效的分析数据就达到了目的。

- 数据仓库，是在数据库已经大量存在的情况下，为了进一步挖掘数据资源、为了决策需要而产生的，它决不是所谓的“大型数据库”。

OLTP vs. OLAP

数据库: [联机事务处理](#) OLTP (OnLine Transaction Processing)

数据仓库: [联机分析处理](#) OLAP (On-Line Analytical Processing)

	OLTP	OLAP
用户	办事员, IT 从业人员	知识工人
功能	日常操作	决策支持
DB 设计	面向应用	面向主题
数据	当前的, 最新的, 细节的, 展平的关系的, 孤立的	历史的, 汇总的, 多维的, 集成的, 加固的
用法	重复	特殊的
访问	读/写 在主键上索引/散列	大量扫描
工作单位	短的, 简单的事务	复杂的查询
访问的记录量	数以十计	数百万
用户数	数千	数百
数据库大小	100MB-GB	100GB-TB
度量	事务吞吐量	查询吞吐量, 响应时间

为什么建立分离的数据仓库？

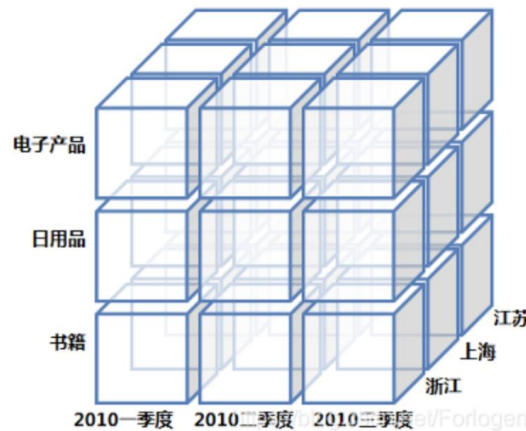
- 为了两个系统的**高性能**
 - 数据库—目的是 OLTP: 存取方法, 索引, 并发控制, 恢复机制
 - 数据仓库—目的是 OLAP: 复杂的 OLAP 查询, 多维视图, 统一
- 不同的功能和不同的数据:
 - **缺少数据**: 决策支持需要历史数据, 通常操作数据库并不维护这些数据
 - **数据统一**: 决策支持需要将来自异种数据源的数据统一 (聚集, 汇总)
 - **数据质量**: 不同的数据源通常使用不同的数据表示, 编码, 和应当遵循的格式

第4-5章: 数据挖掘的数据仓库与OLAP技术

- 什么是数据仓库?
- **多维数据模型**
- 数据仓库结构
- 数据仓库实现
- 从数据仓库到数据挖掘
- 数据立方体的进一步发展

由表和电子数据表到数据方

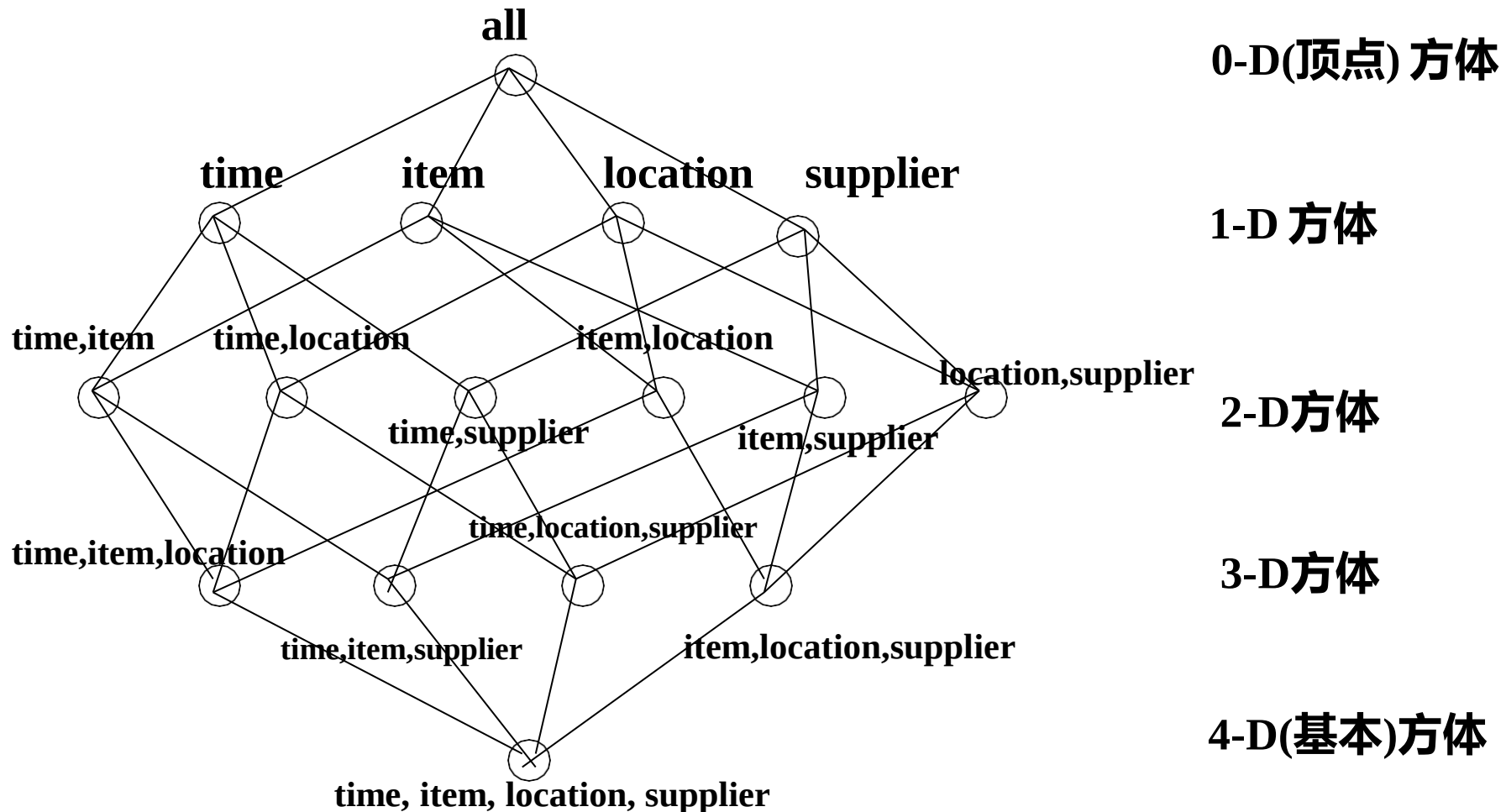
- 数据仓库基于 **多维数据模型**，多维数据模型将数据视为数据方 (data cube)形式



- 数据方是基于事实和维的数据库模型, 并允许由多个维进行观察
 - 维表: 观察数据的一种角度, 上图中产品类别、产地、时间都可以被看作一个维度
 - 事实表: 存放度量值的表, 同时存放了维表的外键, 所有分析所用的数据最终都来自事实表 (外键: 就是一系列数字ID, 它们是通往各个描述性维度表的“门户”或“指针”, 让我们能够从业务角度 (谁、什么、何时、何地) 来分析和理解度量值 (金额、数量等))

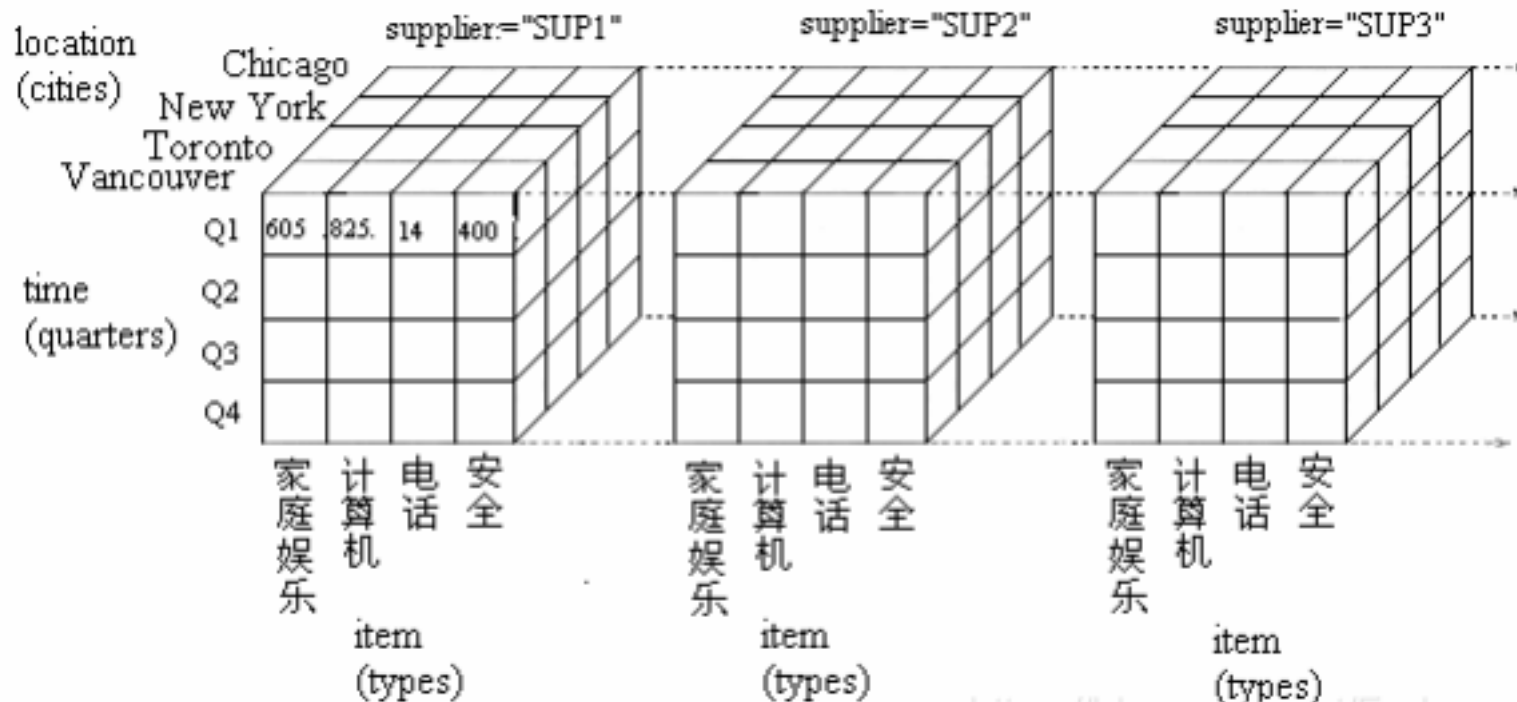
立方体: 方体的格

一个n维的数据的立方体称为基本方体。给定一个维的集合，可以构造一个方体的格，每个都在不同的汇总级或不同的数据子集显示数据，方体的格称为数据立方体。0维方体存放最高层的汇总，称为顶点方体；存放在最底层汇总的方体称为基本方体。



数据立方体只是多维模型的一种形象的说法，它只有三维，但多维数据模型不仅限于三维，它可以是n维的。之所以这么叫是为了让用户更容易想象，方便解释和说明，同时也为了和传统的关系数据库中的二维表进行区分。

我们可以把任意的n维数据立方体看做是 (n-1) 维立方体的序列，比如可以将4-D立方体看做是3-D立方体的序列

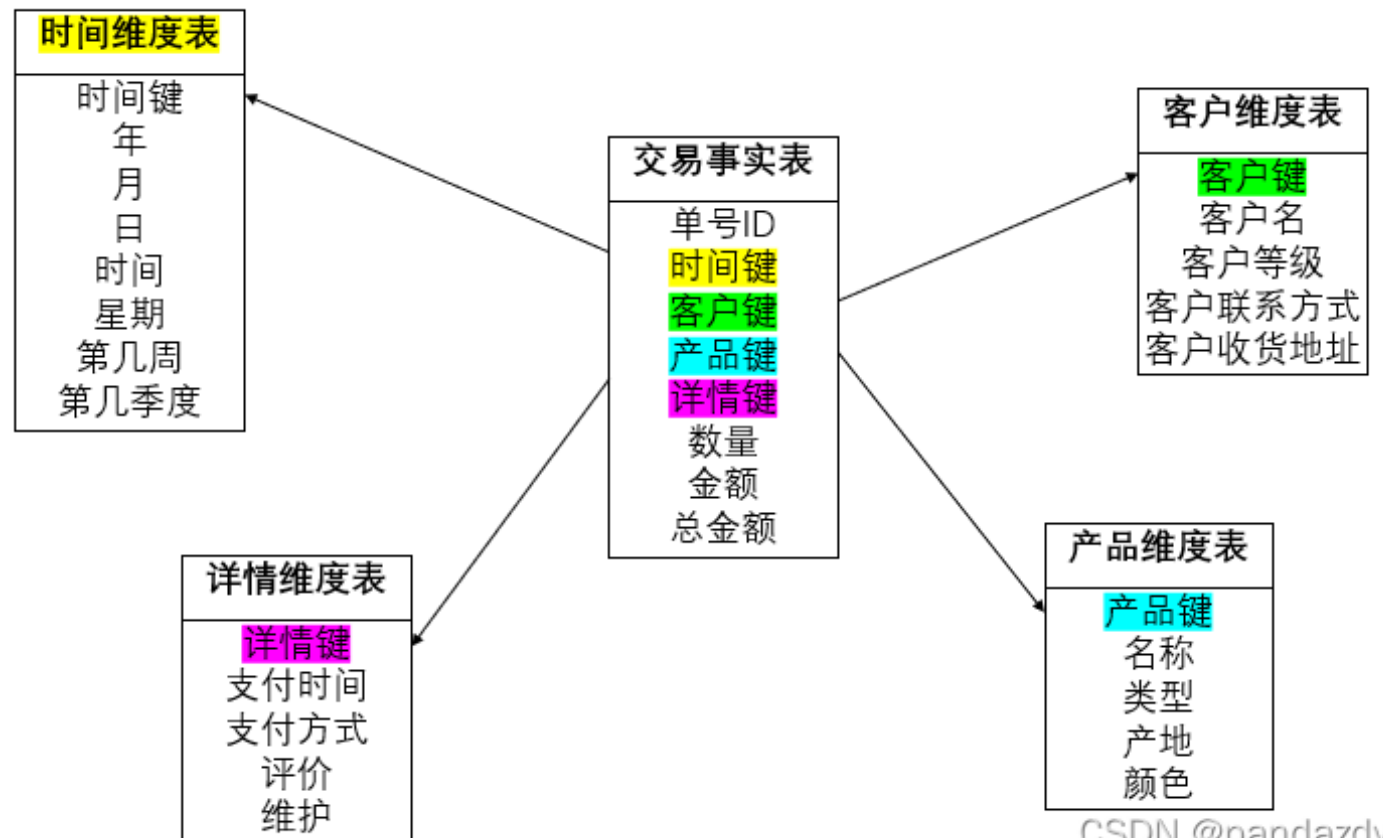


数据仓库的概念建模

- **数据仓库建模:**每个维度对应一个或多个维表, 一个维度对应一个表的是星型模式, 对应多个表的是雪花模式
 - **星型模式:**事实表在中央, 连接一组维表
 - **雪花模式:**星型模式的精炼, 其中一些维分层结构被规范化成一组较小的维表, 形成类似于雪花的形状, 减少冗余
 - **事实星座:**多个事实表共享维表, 可以看作星星的集合, 因此称作星系模式, 或事实星座

星型模式的例子

它是最常见的模式，它包括一个大的中心表（事实表），包含了大批数据但是不冗余；一组小的附属表（维表），每维一个。如下所示，从item、time、branch、location四个维度去观察数据，中心表是Sales Fact Table，包含了四个维表的标识符（由系统产生）和三个度量。



CSDN @pandazdy

事实表充当星型模式的核心。 典型的事实表包含表示业务事件、事务或测量的定量数据或度量。 这些事实通常是数值，例如销售收入、利润或销售数量。

我们可以在下面看到事实表的示例。 它包含可衡量的事实，例如销售单位的价格和销售税。

日期	商店位置	产品类别	数量	单价	销售额	库存	营业税
8/4/2023	CA	尼龙	5	100	500	30	7.75%
8/4/2023	CA	聚酯纤维	7	250	1750	50	7.75%
8/4/2023	PA	尼龙	6	100	600	65	6.00%

维度表是辅助表，为事实表中的数据提供上下文和描述性属性。 它们帮助回答与业务事件相关的“谁”、“什么”、“何时”、“何地”和“如何”等问题。 维度表保存分类数据，例如客户名称、产品类别、时间段和地理位置。

示例：下面的两个维度表提供了上面事实表中产品信息的详细信息。

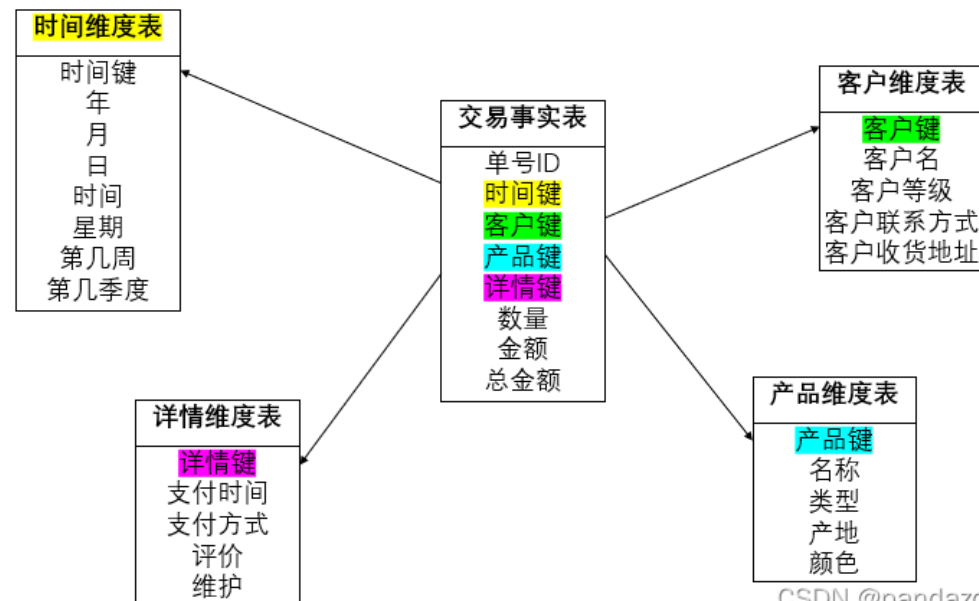
日期维度		
日期键	日期	日
10201	8/4/2023	星期六
10202	8/4/2023	星期六

日期维度表			
店铺规模			
存储密钥	商店名称	城市	州/领地
151	AngAngie'sparel Angie'sparel	洛杉矶	加州
152	AngAngie'sparel Angie'sparel	匹兹堡	宾夕法尼亚

店铺维度表

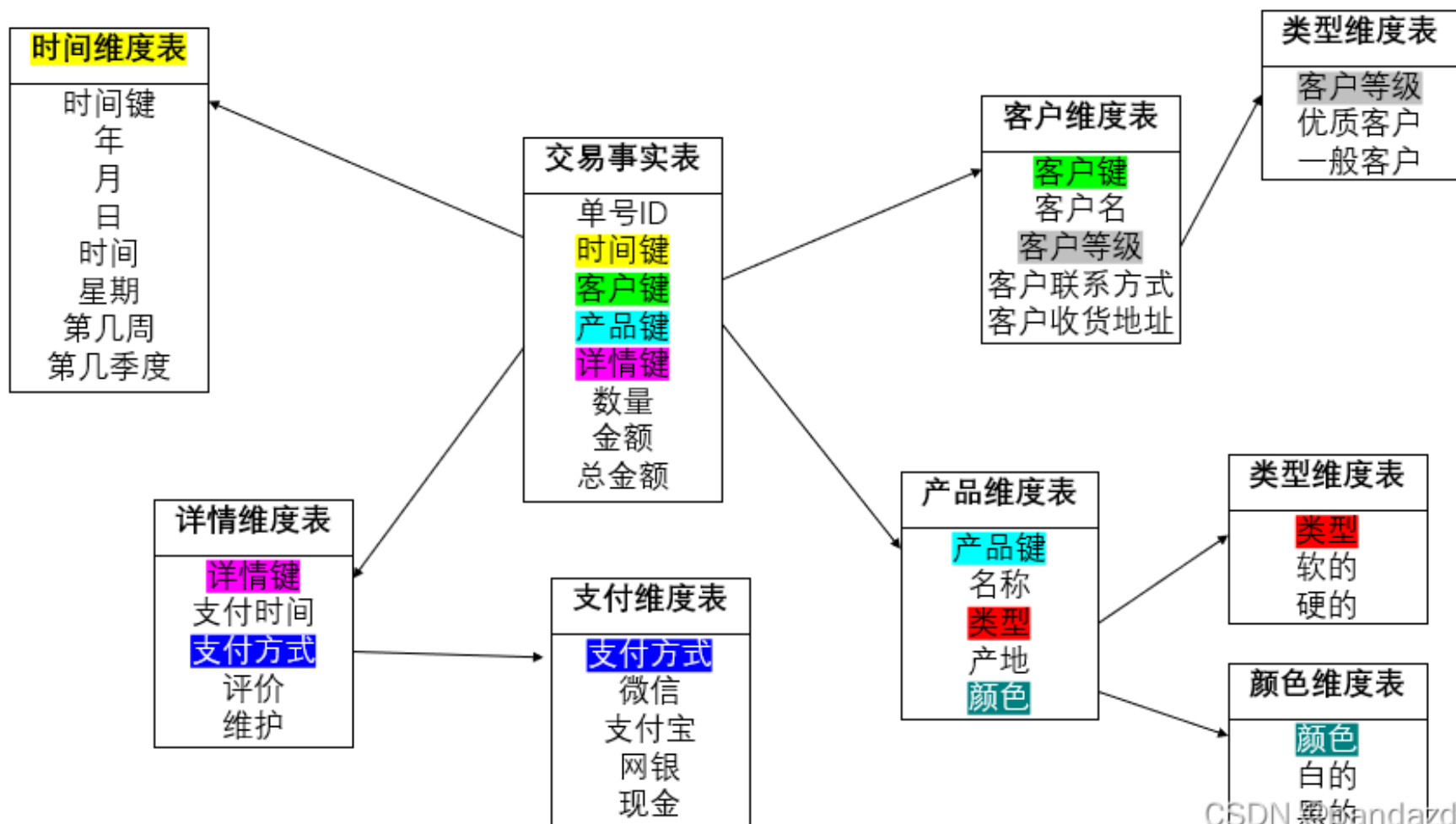
使用星形模式主要有两方面的原因：

- 提高查询的效率。采用星形模式设计的数据仓库的优点是由于数据的组织已经过预处理，主要数据都在庞大的事实表中，所以只要扫描事实表就可以进行查询，而不必把多个庞大的表联接起来，查询访问效率较高，同时由于维表一般都很小，甚至可以放在高速缓存中，与事实表进行连接时其速度较快，便于用户理解；
- 对于非计算机专业的用户而言，星形模式比较直观，通过分析星形模式，很容易组合出各种查询。



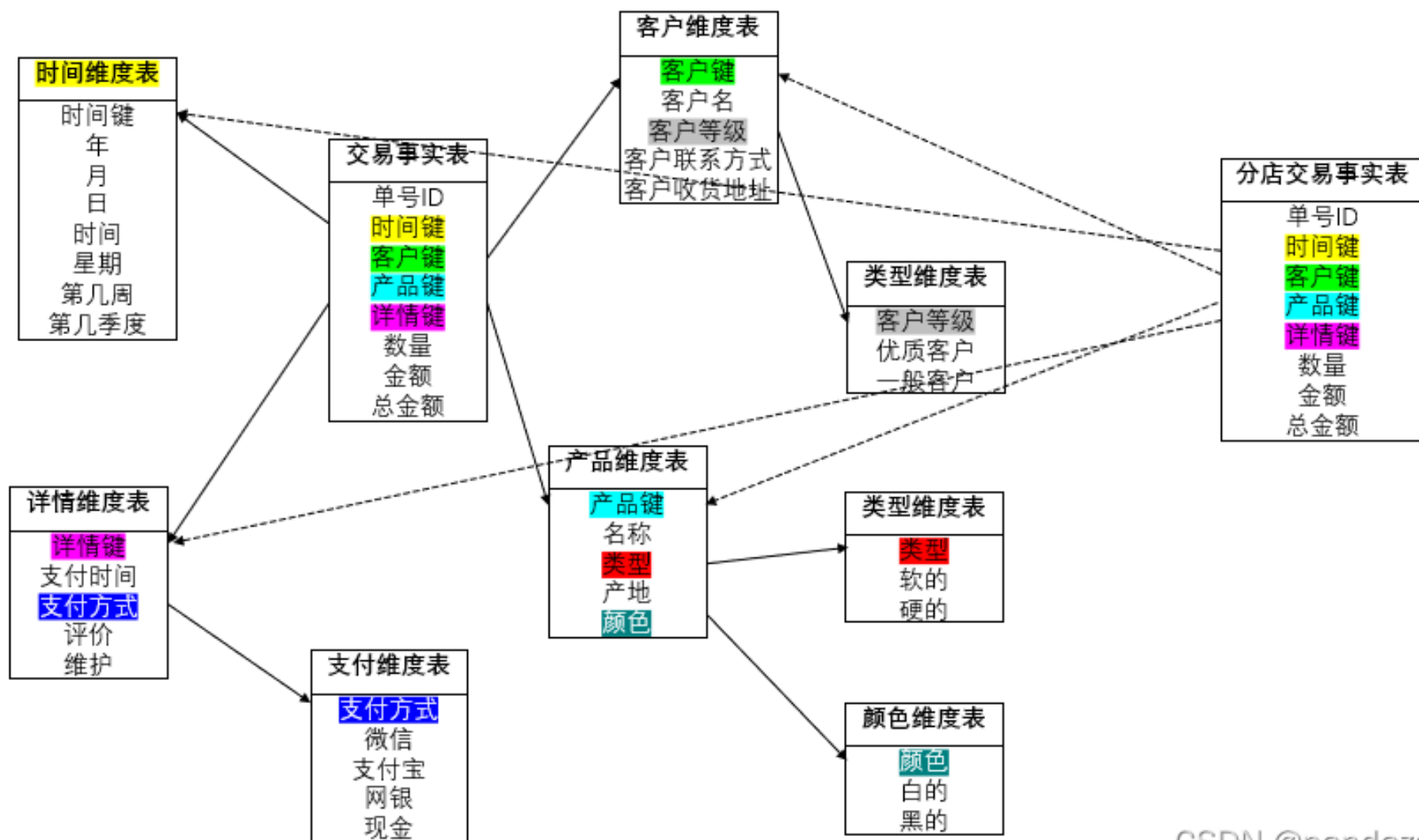
雪花模式的例子

它是星模式的变种，将其中某些表规范化，把数据进一步的分解到附加的表中，形状类似雪花。如下所示，item这个维表被规范化，生成了新的item表和supplier表；同样location也被规范化为location和city两个新的表。



事实星座的例子

允许多个事实表共享维表，可以看作是星形模式的汇集。如下所示，Sales和Shipping两个事实表共享了time、item、location三个维表。



数据挖掘查询语言 DMQL: 语言原语

数据挖掘查询语言 (DMQL) 由 Han、Fu、Wang 等人提出。用于 DBMiner 数据挖掘系统。数据挖掘查询语言实际上是基于结构化查询语言 (SQL, Structured Query Language)。

- 立方体定义 (事实表)

define cube <cube_name> [<dimension_list>]: <measure_list>

- 维定义 (维表)

define dimension <dimension_name> **as**
(<attribute_or_subdimension_list>)

- 特殊情况 (共享维表)

- 第一次, 如 “cube definition”

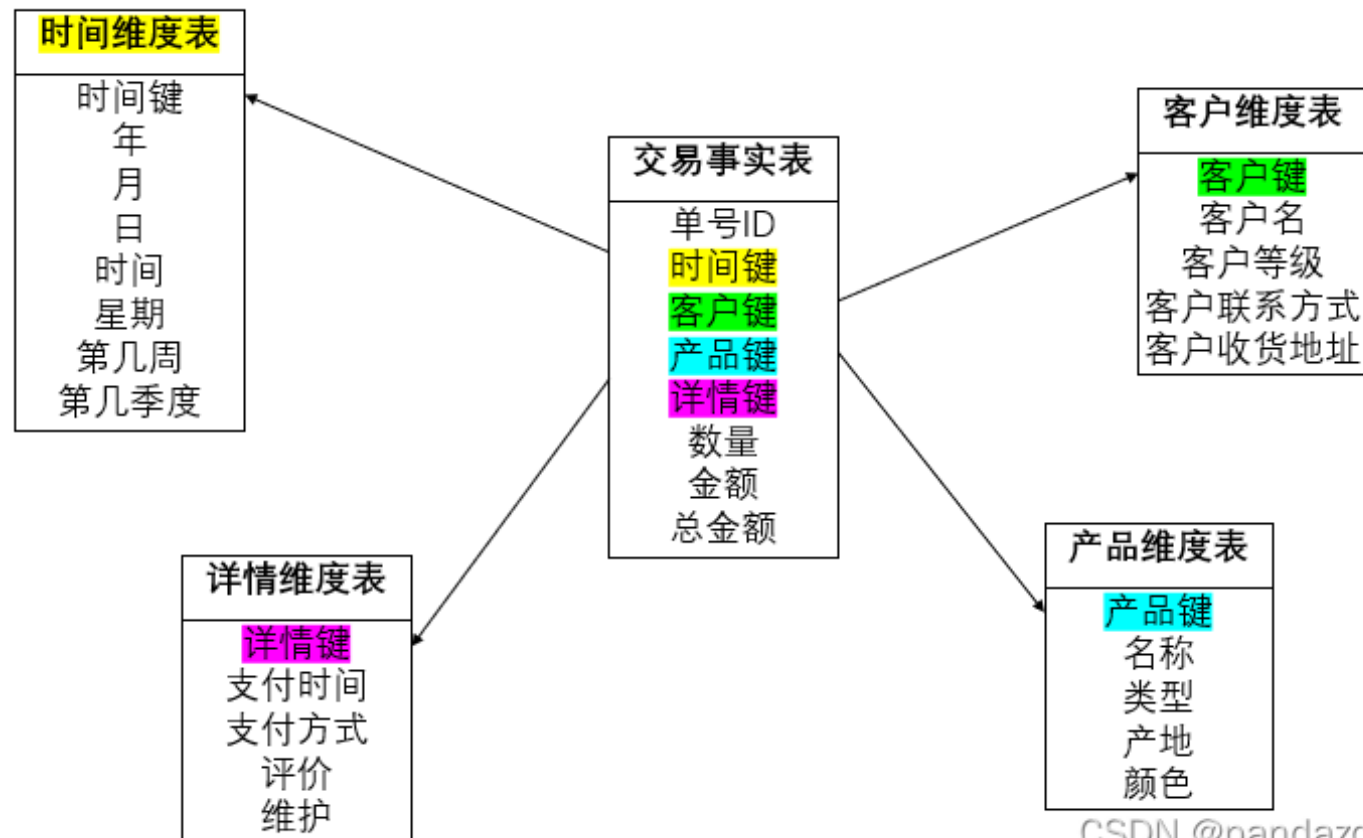
- **define dimension** <dimension_name> **as**
<dimension_name_first_time> **in cube** <cube_name_first_time>

用DMQL定义星型模式

```
define cube sales_star [time, item, branch, location]:  
    dollars_sold = sum(sales_in_dollars), avg_sales =  
        avg(sales_in_dollars), units_sold = count(*)  
  
define dimension time as (time_key, day, day_of_week, month,  
    quarter, year)  
  
define dimension item as (item_key, item_name, brand, type,  
    supplier_type)  
  
define dimension branch as (branch_key, branch_name,  
    branch_type)  
  
define dimension location as (location_key, street, city,  
    province_or_state, country)
```

星型模式的例子

它是最常见的模式，它包括一个大的中心表（事实表），包含了大批数据但是不冗余；一组小的附属表（维表），每维一个。如下所示，从item、time、branch、location四个维度去观察数据，中心表是Sales Fact Table，包含了四个维表的标识符（由系统产生）和三个度量。



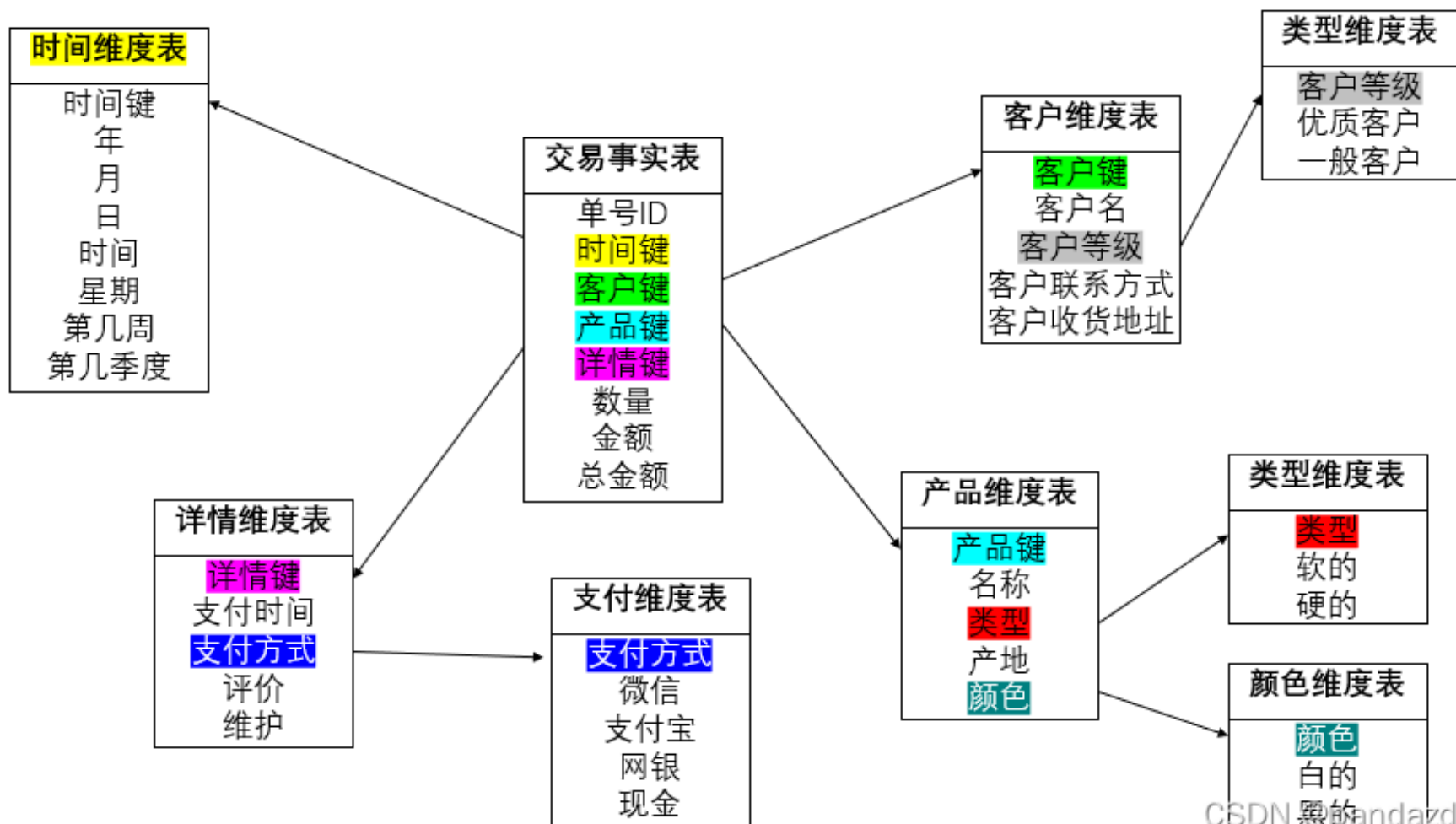
CSDN @pandazdy

用DMQL定义雪花模式

```
define cube sales_snowflake [time, item, branch, location]:  
    dollars_sold = sum(sales_in_dollars), avg_sales =  
        avg(sales_in_dollars), units_sold = count(*)  
define dimension time as (time_key, day, day_of_week, month, quarter,  
    year)  
define dimension item as (item_key, item_name, brand, type,  
    supplier(supplier_key, supplier_type))  
define dimension branch as (branch_key, branch_name, branch_type)  
define dimension location as (location_key, street, city(city_key,  
    province_or_state, country))
```

雪花模式的例子

它是星模式的变种，将其中某些表规范化，把数据进一步的分解到附加的表中，形状类似雪花。如下所示，item这个维表被规范化，生成了新的item表和supplier表；同样location也被规范化为location和city两个新的表。



用DMQL定义事实星座

define cube sales [time, item, branch, location]:

 dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars),
 units_sold = count(*)

define dimension time **as** (time_key, day, day_of_week, month, quarter, year)

define dimension item **as** (item_key, item_name, brand, type, supplier_type)

define dimension branch **as** (branch_key, branch_name, branch_type)

define dimension location **as** (location_key, street, city, province_or_state, country)

define cube shipping [time, item, shipper, from_location, to_location]:

 dollar_cost = sum(cost_in_dollars), unit_shipped = count(*)

define dimension time **as** time **in cube** sales

define dimension item **as** item **in cube** sales

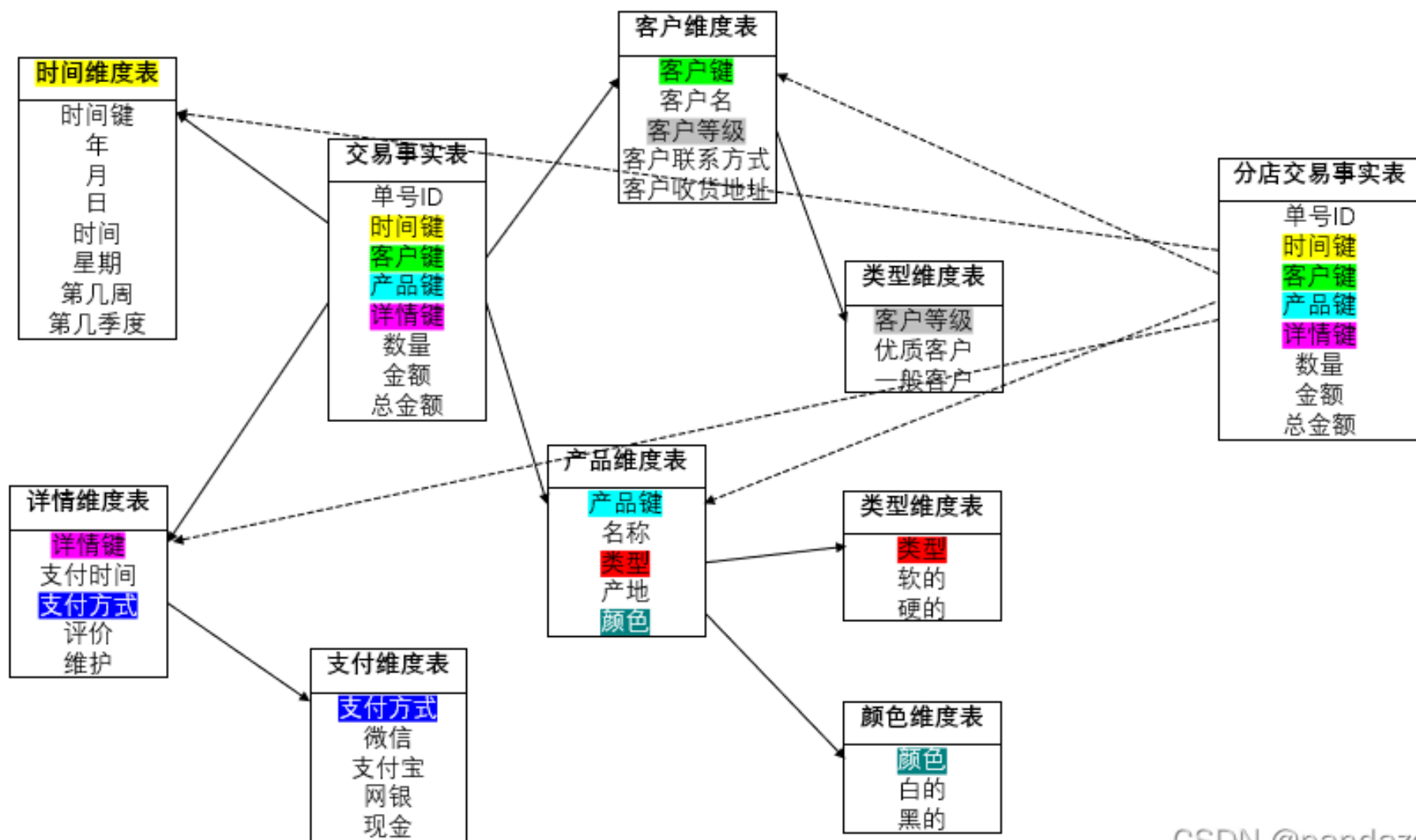
define dimension shipper **as** (shipper_key, shipper_name, location **as** location **in cube** sales, shipper_type)

define dimension from_location **as** location **in cube** sales

define dimension to_location **as** location **in cube** sales

事实星座的例子

允许多个事实表共享维表，可以看作是星形模式的汇集。如下所示，Sales和Shipping两个事实表共享了time、item、location三个维表。



数据立方体框架和工具

Datacube是一个基于Java开发的开源框架，旨在为用户提供一个灵活且高效的平台来构建和操作数据立方体。该框架的核心设计理念是模块化和可扩展性，这意味着开发者可以根据实际需求选择合适的数据库后端进行集成。

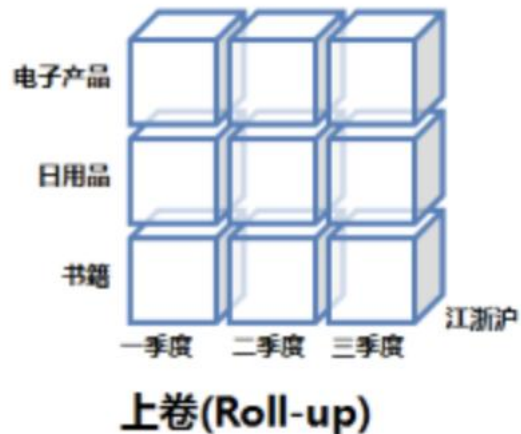
目前，Datacube支持包括MySQL、PostgreSQL以及Hadoop HDFS在内的多种插件式数据库后端，这使得它能够适应不同规模和类型的数据源。

Hive是一个基于Hadoop的数据仓库工具，用于处理和分析大规模结构化数据。

典型的OLAP操作

- 上卷(Roll up)/上钻 (drill-up): 汇总数据
- 下钻(Drill down)/下卷 (roll down): 上卷的逆操作
- 切片(Slice)和切块 :
 - 投影和选择
- 转轴(Pivot)/旋转 (rotate):
 - 调整数据方, 目视操作, 3D 到 2D 平面.
- 其它操作
 - 钻过(drill across): 涉及多个事实表
 - 钻透(drill through): 通过数据方的最底层, 到它背后的关系表 (使用 SQL)

OLAP 操作: 上卷和下钻

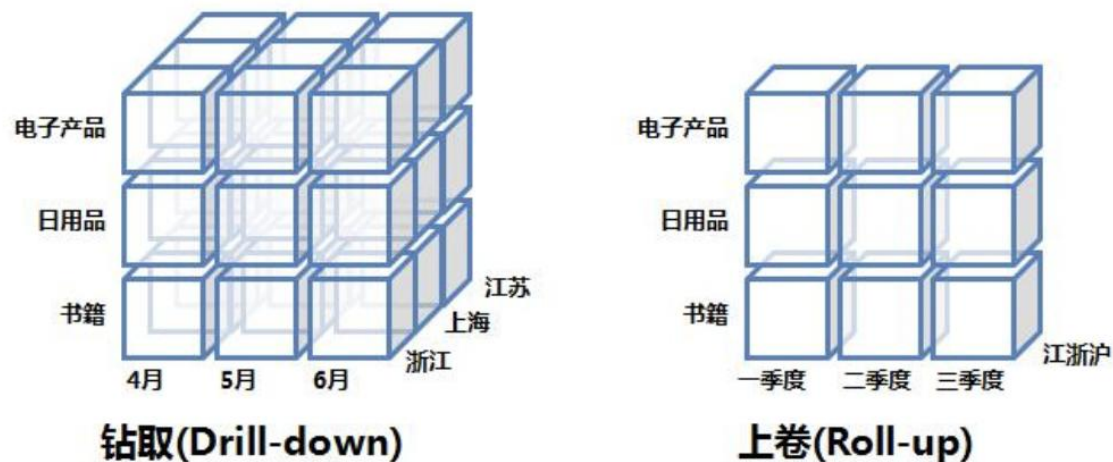


上卷 (Roll-up) : 钻取的逆操作, 即从细粒度数据向高层的**聚合**, 如将江苏省、上海市和浙江省的销售数据进行汇总来查看江浙沪地区的销售数据, 如上图。



钻取/下钻 (Drill-down) : 在维的不同层次间的变化, 从上层降到下一层, 或者说是将汇总**数据拆分**到更细节的数据, 比如通过对2010年4月的总销售数据进行钻取来查看2010年4月每日的消费数据, 如上图; 当然也可以钻取浙江省来查看杭州市、宁波市、温州市.....这些城市的销售数据。

上卷可以理解为"无视"某些维度；下钻则是指将某些维度进行细分。下图逻辑上展示了上卷和下钻操作：

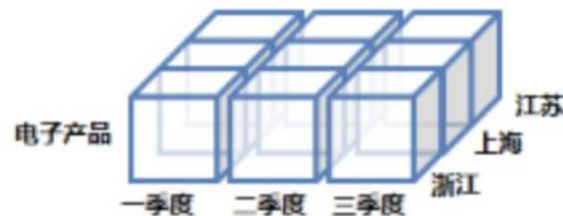


这两种操作的SQL模拟语句如下，主要是对GROUP BY语句做工作：

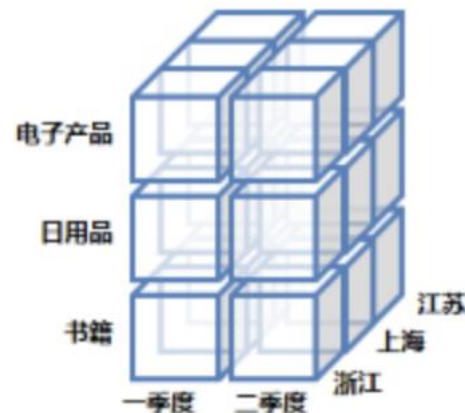
```
1  # 上卷
2  SELECT Locates.地区, Products.分类, SUM(数量)
3  FROM Sales, Products, Locates
4  WHERE Sales.Locate_key = Locates.Locate_key
5         AND Sales.Product_key = Products.Product_key
6  GROUP BY Locates.地区, Products.分类
7
8  # 下钻
9  SELECT Locates.地区, Dates.季度, Products.分类, SUM(数量)
10 FROM Sales, Dates, Products, Locates
11 WHERE Sales.Date_key = Dates.Date_key
12        AND Sales.Locate_key = Locates.Locate_key
13        AND Sales.Product_key = Products.Product_key
14 GROUP BY Dates.季度.月份, Locates.地区, Products.分类
```

OLAP 操作:切片和切块

- 切片 (Slice)：选择维中特定的值进行分析，比如只选择电子产品的销售数据，或者2010年第二季度的数据。
- 切块 (Dice)：选择维中特定区间的数据或者某批特定值进行分析，比如选择2010年第一季度到2010年第二季度的销售数据，或者是电子产品和日用品的销售数据。

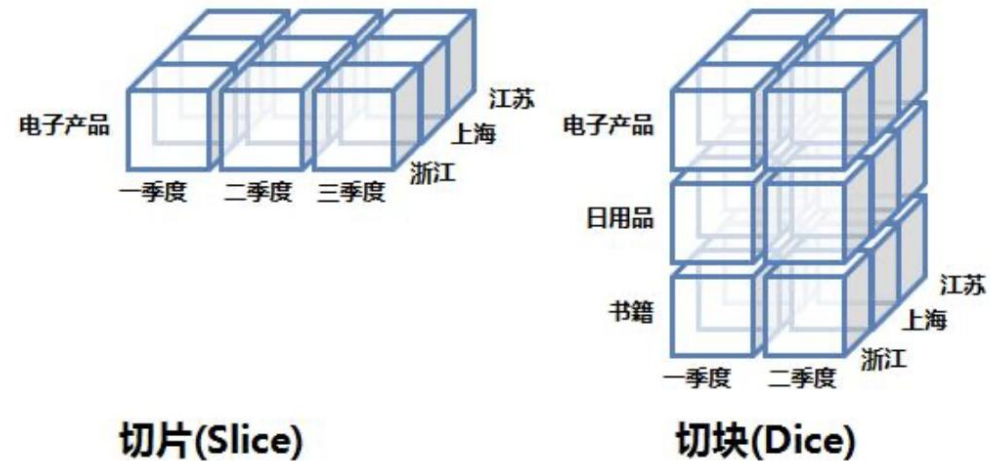


切片(Slice)



切块(Dice)

在数据立方体的某一维度上选定一个维成员的操作叫切片，而对两个或多个维执行选择则叫做切块。下图逻辑上展示了切片和切块操作：

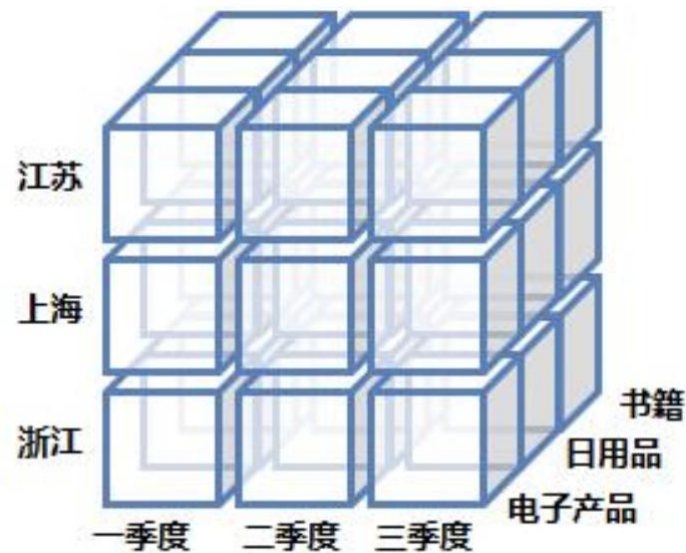


这两种操作的SQL模拟语句如下，主要是对WHERE语句做工作：

```
1  # 切片
2  SELECT Locates.地区, Products.分类, SUM(数量)
3  FROM Sales, Dates, Products, Locates
4  WHERE Dates.季度 = 2
5         AND Sales.Date_key = Dates.Date_key
6         AND Sales.Locate_key = Locates.Locate_key
7         AND Sales.Product_key = Products.Product_key
8  GROUP BY Locates.地区, Products.分类
9
10 # 切块
11 SELECT Locates.地区, Products.分类, SUM(数量)
12 FROM Sales, Dates, Products, Locates
13 WHERE (Dates.季度 = 2 OR Dates.季度 = 3) AND (Locates.地区 = '江苏' OR Locates.地区 = '上海')
14         AND Sales.Date_key = Dates.Date_key
15         AND Sales.Locate_key = Locates.Locate_key
16         AND Sales.Product_key = Products.Product_key
17 GROUP BY Dates.季度, Locates.地区, Products.分类
```

OLAP 操作: 转轴

- 转轴(Pivot)/旋转 (rotate):
 - 旋转 (Pivot) : 即维的位置的互换, 就像是二维表的行列转换, 如图中通过旋转实现产品维和地域维的互换。



其他操作

■ 其它操作

- 钻过(drill across): 涉及多个事实表
- 钻透(drill through): 通过数据方的最底层, 到它背后的关系表 (使用 SQL)
- 统计计算
 - 比率、方差; 增长率
- 分析建模, 等

其他操作

■ 钻过(drill across): 涉及多个事实表

- 多个事实表之间进行查询和关联，以便在同一个报表中整合来自不同业务过程的数据；
- 如何工作？共享一致性维度：两个事实表必须共享一些通用的维度（如时间维度、产品维度、地区维度）；按维度关联：系统会基于这些共享的维度（例如，按产品键和月份键）将两个事实表的数据进行连接（**Join**）或关联，然后在一个结果集中同时展示来自不同事实表的度量值（如销售额和广告成本）

其他操作

- **钻过(drill across): 涉及多个事实表**
 - 简单比喻：看完公司的销售报告（来自销售事实表），又想看同一时期的市场花费报告（来自市场活动事实表）。钻过操作让你能把这两份报告并排放在一起，按相同的产品线和月份进行对比。

其他操作

■ 钻过(drill across): 涉及多个事实表

```
SELECT
    dim_product.product_name,
    dim_time.month,
    SUM(sales_fact.sales_amount) AS total_sales, -- 来自销售事实表
    SUM(marketing_fact.cost_amount) AS total_marketing_cost -- 来自市场活动事实表
FROM
    sales_fact
JOIN
    marketing_fact ON sales_fact.product_key = marketing_fact.product_key
                  AND sales_fact.time_key = marketing_fact.time_key
JOIN
    dim_product ON sales_fact.product_key = dim_product.product_key
JOIN
    dim_time ON sales_fact.time_key = dim_time.time_key
GROUP BY
    dim_product.product_name, dim_time.month;
```

其他操作

- **钻透(drill through)**: 通过数据方的最底层, 到它背后的关系表 (使用 SQL)
 - 当分析师发现某个汇总数据异常（如某个月份销售额异常高），需要查明“到底是谁、在什么时候、买了什么东西”导致了这种情况时，就需要钻透；
 - 如何工作？用户在**OLAP**工具中点击一个汇总数据单元格（例如：**2023年10月，产品A，总销售额\$100,000**）；执行“钻透”操作；系统在后台生成**SQL**查询语句，直接查询支撑这个数据立方体的最底层关系型数据库表（即事实表），返回构成这**\$10万**的所有原始销售单据。

其他操作

- **钻透(drill through)**: 通过数据方的最底层, 到它背后的关系表 (使用 SQL)
 - 简单比喻: 看一份年度财务报表的总收入数字 (汇总数据)。钻透就像是双击这个总收入数字, 电脑立刻弹出了一份详细的、列出本年所有收入记录的清单 (明细数据)。
 - 与技术的关系: 这是**OLAP**系统与底层**ROLAP** (关系型**OLAP**) 架构紧密集分的体现。钻透的本质是自动生成并执行**SQL**查询, 返回到数据仓库的关系表中去获取明细。

其他操作

- **统计计算**

- **比率、方差；增长率**

- OLAP工具不仅提供基本的聚合函数（如SUM, COUNT, AVG），还内置了高级统计函数，用于更深入的数据分析；这些计算将数据分析从“发生了什么”提升到了“为什么会发生”和“趋势如何”的层面，是商业智能的核心。

其他操作

■ 分析建模

- 利用OLAP系统中的数据，构建复杂的数学模型来预测未来、模拟场景或进行深度挖掘。这通常是OLAP功能的延伸，有时需要与专业的数据挖掘/统计工具集成。
- 预测 (Forecasting): 基于历史数据趋势，预测未来的销售额、需求量等。常用方法包括时间序列分析、回归分析等。
- 假设分析 (What-If Analysis): 模拟不同业务决策可能带来的结果。
- 示例: 建立一个模型，模拟“如果产品降价10%，销售额需要增长多少才能保持总利润不变？”
- 数据挖掘 (Data Mining): 应用聚类、分类、关联规则等算法，发现隐藏模式。
- 示例: 购物篮分析（通过关联规则发现“买了A产品的顾客很大概率也会买B产品”）。
- 评分 (Scoring): 根据模型为客户、潜在线索等进行打分，例如“客户流失风险评分”、“信用评分”等。

第4-5章: 数据挖掘的数据仓库与OLAP技术

- 什么是数据仓库?
- 多维数据模型
- 数据仓库结构与设计
- 数据仓库实现
- 从数据仓库到数据挖掘
- 数据立方体的进一步发展

数据仓库结构

- 按照数据流入流出的过程，数据仓库架构可分为三层——源数据、数据仓库、数据应用。



数据仓库结构

•**源数据层** (ODS)：操作性数据(Operational Data Store)，是作为数据库到数据仓库的一种过渡，ODS的**数据结构**一般与数据来源保持一致，可以增加字段用来进行数据管理，存储的历史数据只是只读的，提供业务系统查询使用，而且ODS的**数据周期**一般比较短。ODS的数据为后一步的数据处理做准备。

•**数据仓库层** (DW)：数据仓库(Data Warehouse)，是数据的归宿，这里保持这所有的从ODS到来的数据，并长期保存，而且这些数据不会被修改，DW层的数据应该是一致的、准确的、干净的数据，即对源系统数据进行了清洗（去除了杂质）后的数据。

•**数据应用层** (DA)：数据应用(Data Application)，为了特定的应用目的或应用范围，而从数据仓库中独立出来的一部分数据，也可称为部门数据或主题数据，该数据面向应用。如根据报表、专题分析需求而计算生成的数据

数据仓库设计

- 数据仓库设计中，必须考虑四种视图
 - 自顶向下视图
 - 选择数据仓库所需的有关信息
 - 数据源视图
 - 揭示（操作）数据库系统捕获、存储、和管理的信息
 - 数据仓库视图
 - 由事实表和维表组成
 - 商务查询视图
 - 从最终用户的角度透视数据仓库中的数据

数据仓库设计

■ 自顶向下视图（Top-Down View）

■ 选择数据仓库所需的有关信息

- 核心要义：“我们需要什么？” 这是一个业务驱动的视角。它从企业高层的战略目标和业务需求出发，而不是从现有的数据源出发。
- 主要内容：
 - ✓ 确定范围：明确数据仓库要解决哪些关键的商业问题（例如，提升客户满意度、分析销售趋势、优化供应链效率）。
 - ✓ 识别主题域：将业务需求划分为不同的主题领域（**Subject Areas**），如销售、客户、产品、财务等。每个主题域将成为数据仓库中的一个核心模块。
 - ✓ 定义关键绩效指标 (**KPI**)：与业务部门共同确定衡量业务健康度的核心指标，如客户生命周期价值、月度经常性收入、库存周转率等。
 - ✓ 目标状态架构：规划理想状态下的企业级数据仓库架构，包括数据流、数据市场(**Data Marts**)的划分等。

数据仓库设计

■ 数据源视图（Data Source View）

■ 揭示（操作）数据库系统捕获、存储、和管理的信息

- 核心要义：“我们有什么？” 这是一个技术驱动的视角。它深入探查现有的操作型系统（源系统），了解数据的“家底”。
- 主要内容：
 - ✓ 源系统清单： 识别所有需要从中抽取数据的源系统，如**ERP**、**CRM**、财务软件、日志文件、**Excel**表格等。
 - ✓ 数据结构分析： 分析每个源系统的数据库结构（表、字段、数据类型）、数据量、更新频率。
 - ✓ 数据质量评估： 初步评估数据的质量，发现是否存在脏数据、重复数据、不一致、缺失值等问题。
 - ✓ 映射关系梳理： 初步思考如何将分散在不同源系统中的、格式不一的数据，转换和集成为目标数据仓库中的一致格式。

数据仓库设计

■ 数据仓库视图（Data Warehouse View）

■ 由事实表和维表组成

- 核心要义：“我们怎么存？”这是一个建模师/架构师的视角。它是前两种视图的交汇点，将业务需求映射到具体的技术实现。
- 主要内容：
 - ✓ 逻辑模型设计：选择建模范式（如经典的维度建模——星型模式、雪花模式），设计事实表（存储度量值，如销售额）和维度表（存储描述信息，如时间、产品、客户）。
 - ✓ 物理模型设计：基于逻辑模型，在具体的**DBMS**（数据库管理系统）中实现。包括确定表结构、主外键、索引、分区策略、存储引擎等，以优化查询性能和存储效率。
 - ✓ 数据粒度定义：明确事实表中每行记录的详细程度，是“每笔交易一条记录”还是“每天汇总一条记录”。这是设计中最关键的决策之一。
 - ✓ 汇总表与聚合设计：为了提升常用查询的性能，预先设计并存储一些汇总数据（**Aggregations**）。

数据仓库设计

■ 商务查询视图（Business Query View）

■ 从最终用户的角度透视数据仓库中的数据

- 核心要义：“我们怎么用？” 这是一个最终用户的视角。它关注的是如何将数据仓库中复杂的数据以简单、直观的方式呈现给业务分析师、管理层等用户。
- 主要内容：
 - ✓ 前端工具选择： 确定用户访问数据的工具，如**Tableau**、**Power BI**、**Qlik**、**Excel**等BI报表工具。
 - ✓ 语义层/数据模型构建： 在BI工具中建立友好的业务逻辑层，将复杂的表连接和计算逻辑封装成业务用户能理解的“业务对象”（如利润、活跃客户），屏蔽底层技术复杂性。
 - ✓ 报表与仪表盘设计： 设计与业务**KPIs**直接关联的固定报表、交互式仪表盘、即席查询(**Ad-hoc Query**)界面。
 - ✓ 安全与权限管理： 定义不同用户角色和组的数据访问权限，确保数据安全。

数据仓库设计

这四种视图是一个有机整体，贯穿了数据仓库从规划到落地的全过程：

- 自顶向下视图和数据源视图是输入，分别定义了目标和现状。
- 数据仓库视图是核心处理过程，它在目标和现状之间找到平衡点，设计出最佳的数据存储结构。
- 商务查询视图是输出，它将数据仓库的价值交付给最终用户，驱动业务决策。

一个常见比喻：

- 自顶向下视图 = 建筑蓝图（客户想要一栋什么样的房子）
- 数据源视图 = 地基和建材（工地环境和现有材料）
- 数据仓库视图 = 建筑结构与施工（工程师如何设计并建成这栋房子）
- 商务查询视图 = 精装修与家具电器（如何让房子变得舒适、宜居、方便使用）

数据仓库设计过程

- 自顶向下, 自底向上方法或二者的结合
 - 自顶向下: 由总体设计和规划开始 (成熟)
 - 自底向上: 由实验和原型开始 (快速)
- 软件工程的观点
 - 瀑布式: 在进行下一步之前, 每一步都进行结构化和系统的分析
 - 螺旋式: 功能渐增的系统的快速产生, 相继版本之间的间隔很短, 快速转向
- 典型的数据仓库设计过程
 - 选取待建模的商务处理, 例如, 订单, 发票, 库存等.
 - 选取商务处理的粒度 (**原子层数据**), 例如, 单个事务、一天的快照等
 - 选取用于每个事实表记录的维, 如, 时间、商品、顾客、供应商、仓库、事务类型和状态 等
 - 选取将安放在事实表中的度量. 典型的度量是可加的数值量, 如 *dollars_sold*和*units_sold*

OLAP 服务器结构

■ 多维 OLAP (MOLAP)

- 基于数组的多维存储引擎 (稀疏矩阵技术)
- 对预计算的汇总数据快速索引

1. MOLAP(Multidimensional Online Analytical Processing)

MOLAP架构会生成一个新的多维数据集，也可以说是构建了一个**实际数据立方体**。其架构如下图所示：



在该立方体中，每一格对应一个直接地址，且常用的查询已被预先计算好。因此每次的查询都是非常快速的，但是由于立方体的更新比较慢，所以是否使用这种架构得具体问题具体分析。

OLAP 服务器结构

■ 关系OLAP (ROLAP)

- 使用关系或扩充关系的 DBMS 存放和管理仓库数据, 使用OLAP中间件支持其它部分
- 包含一个优化的 DBMS 后端, 聚集导航逻辑的实现, 以及附加的工具和服务
- 较大的可伸缩性

2. ROLAP(Relational Online Analytical Processing)

ROLAP架构并不会生成实际的多维数据集, 而是使用星形模式以及多个关系表对数据立方体进行模拟。其架构如下图所示:



显然, 这种架构下的查询没有MOLAP快速。因为ROLAP中, 所有的查询都是被转换为SQL语句执行的。而这些SQL语句的执行会涉及到多个表之间的JOIN操作, 没有MOLAP速度快。

OLAP 服务器结构

- **混合 OLAP (HOLAP)**
 - 弹性, 底层: 关系的, 高层: 数组.

3. HOLAP(Hybrid Online Analytical Processing)

这种架构综合参考MOLAP和ROLAP而采用一种混合解决方案，将某些需要特别提速的查询放到MOLAP引擎，其他查询则调用ROLAP引擎。

第2章: 数据挖掘的数据仓库与OLAP技术

- 什么是数据仓库?
- 多维数据模型
- 数据仓库结构
- **数据仓库实现**
- 从数据仓库到数据挖掘
- 数据立方体的进一步发展

数据仓库之ETL

建立OLAP应用之前，我们要想办法把各个独立系统的数据抽取出来，经过一定的转换和过滤，存放到一个集中的地方，成为数据仓库。这个抽取，转换，加载的过程叫ETL（Extract， Transform， Load），目的是将企业中分散、零乱、标准不统一的数据整合到一起。

ETL是数据仓库的流水线，也可以认为是数据仓库的血液，它维系着数据仓库中数据的新陈代谢，而数据仓库日常的管理和维护工作的大部分精力就是保持ETL的正常和稳定。



数据仓库之ETL

1、抽取

抽取的数据源分为结构化数据、非结构化数据，半结构化数据。

结构化一般采用JDBC、数据库日志方式，非结构或半结构化数据会建通文件变动。

抽取方式：

- 全量抽取：适用于数据量小且不容易判断其数据发生改变的诸如关系表，维度表，配置表等，一般用于初始化数据。增量抽取：适用于数据量大，为了节省抽取时间而采用的抽取策略，一般用于数据更新

数据仓库之ETL

2、清洗

- **空值处理：将空值替换为特定值或直接过滤掉**
- **验证数据正确性：把不符合业务含义的数据做统一处理**
- **规范数据格式：比如把所有日期都规范成YYYY-MM-DD的格式**
- **数据转码：把一个源数据中用编码表示的字段通过关联编码表转换成代表其真实意义的值**
- **数据标准统一：比如在源数据中表示男女的方式有很多种，在抽取的时候直接根据模型中定义的值做转化。**
- **注：主要是针对非结构化或半结构化数据**

数据仓库之ETL

3、转换和加载

- 转换：用ODS中的增量或者全量数据来刷新DW中的表
- 加载：每insert数据到一张表都可以称为数据加载，就是将处理完的数据导入到对应的目标源里

ETL工具

结构化数据ETL工具：

Sqoop、Kettle、Datastage、Informatica、Kafka

非|半结构化数据ETL工具：

Flume、Logstash

数据方的有效计算

- 多维数据分析的核心是有效计算多个维度的组合上的聚集。按SQL术语，这些聚集称为group-by。
- 数据方可以视为方体的格
 - 最下面的方体是基本方体，它包含最细粒度的数据
 - 最上面的 (顶点) 方体只包含一个单元（即所有维度的聚集总和）
 - 具有L层的n-D数据方包含多少个方体？

数据方的有效计算

- 具有L层的n-D数据方包含多少个方体？（n表示维数）

公式: $T = \prod_{i=1}^n (L_i + 1)$

- T : 方体总数量
- n : 维度的总数
- i : 维度索引 (从1到n)
- L_i : 第 i 个维度所具有的概念层次的数量
- $\prod$: 连乘符号, 代表将所有 $(L_i + 1)$ 相乘

我们有一个数据立方体，它包含 2 个维度：

1. **时间维度**：具有 日 -> 月 -> 年 这个完整的层次结构。
2. **地区维度**：具有 城市 -> 国家 这个完整的层次结构。

分步计算

现在我们将这两个维度代入公式：

1. 时间维度分析：

- 可分组级别：日，月，年 → 共 3 个级别。
- 路径数量 $Options_{time} = (3\text{个分组级别}) + (1\text{个“忽略”选项}) = 4$

2. 地区维度分析：

- 可分组级别：城市，国家 → 共 2 个级别。
- 路径数量 $Options_{region} = (2\text{个分组级别}) + (1\text{个“忽略”选项}) = 3$

3. 计算总方体数 (T)：

由于两个维度的选择是独立的，根据乘法原理：

$$T = Options_{time} \times Options_{region} = 4 \times 3 = 12$$

因此，这个包含**时间**（3级层次）和**地区**（2级层次）两个维度的数据立方体，总共可以生成 12 个不同的方体。

每个方体都是两个维度选择的唯一组合：

- 时间维度的4种选择：{忽略, 按年分组, 按月分组, 按日分组}
- 地区维度的3种选择：{忽略, 按国家分组, 按城市分组}

例如：

- GROUP BY 年，国家 是一个方体。
- GROUP BY 月，城市 是另一个方体。
- GROUP BY 年 (忽略地区) 也是一个方体。
- GROUP BY () (同时忽略时间和地区，计算全局总和) 是第12个方体。

数据方的有效计算

■ 数据方的物化(Materialization)

- 物化指的是预先计算并存储结果的过程。
- 对于数据立方体来说：物化就是指预先计算好某个方体（Cuboid）（即针对某些维度组合的 GROUP BY 查询结果），并将其作为一个实际的、物理上的表或视图存储在硬盘上。**相反的操作：**每次用户查询时，都临时从最原始的事实表中进行汇总计算。
- 举例：
 - ✓ 物化：就像是你每天晚上提前把第二天要卖的盒饭都做好，放进冰箱。顾客点单时，你直接拿出来热一下就能上菜，速度非常快，但需要占用冰箱空间，而且如果没人买，你就浪费了精力。
 - ✓ 不物化：就像是你只有一个菜谱和一堆食材。顾客点单时，你才现切现炒。非常节省冰箱空间，但做菜速度很慢，让顾客等得久。

数据方的有效计算

■ 数据方的物化(Materialization)

- 物化每一个方体 (全物化), 不物化任何方体(不物化), 或物化某些方体(部分物化)
- 全物化 (**Full Materialization / Pre-computation**)
- 做法: 计算并存储数据立方体中所有可能的方体 (即所有维度组合的 **GROUP BY** 结果)。
- 优点: 查询性能极快。任何用户的任何聚合查询, 都可以直接从物化好的结果中读取, 无需临时计算。
- 缺点: 存储成本巨大。一个 n 维的立方体有 2^n 个方体。当维度 n 很大时, 所需的存储空间会呈指数级增长, 可能达到原始数据的数十倍甚至上百倍。维护更新这些方体 (如增量更新) 的开销也极大。
- 适用场景: 维度较少、数据量相对不大、且对查询响应速度要求极高的场景。

数据方的有效计算

■ 数据方的物化(Materialization)

- 物化每一个方体 (全物化), 不物化任何方体(不物化), 或物化某些方体(部分物化)
- 不物化 (No Materialization)
- 做法: 只存储最细粒度的基本方体 (**Base Cuboid**)。任何用户的聚合查询, 都需要系统临时从基本方体出发, 执行一次完整的 **GROUP BY** 计算。
- 优点: 极其节省存储空间。只存一份最详细的数据。
- 缺点: 查询性能极差。尤其是当数据量庞大时, 每次复杂的聚合查询都可能需要耗时数分钟甚至数小时, 用户体验非常糟糕。
- 适用场景: 主要用于理论概念, 实际生产中很少采用。或用于存储原始数据, 而非用于直接查询。

数据方的有效计算

- **数据方的物化(Materialization)**
 - 物化每一个方体 (全物化), 不物化任何方体(不物化), 或物化某些方体(部分物化)
 - 部分物化 (**Partial Materialization**)
 - 做法：这是一种折中策略。系统根据一定的规则和算法，选择性地物化一部分（而非全部）方体。
 - 目标：在查询性能和存储成本之间找到一个最佳平衡点。用有限的存储空间，最大化地提升最常用查询的速度。
 - 这是绝大多数商业系统采用的策略

数据方的有效计算

- 数据方的物化(Materialization)
 - 物化方体的选择
 - 基于大小, 共享, 访问频率, 等.

选择依据	描述	例子
大小 (Size)	优先物化“小”的方体。一个方体分组聚合后包含的行数越少，存储它成本越低，查询扫描它的速度也越快。	物化 GROUP BY 年, 国家（可能只有几百行），而不是 GROUP BY 日, 城市（可能有几百万行）。
访问频率 (Access Frequency)	优先物化被频繁查询的方体。这是最重要的依据之一。经常被访问的方体物化后收益最大。	如果管理层每天都看《各品牌月度销售额报表》，那么就物化 GROUP BY 品牌, 月 这个方体。
共享 (Sharing)	优先物化那些能被多个查询共享的方体。物化一个“高层”方体，可以用来回答更多“低层”查询。	物化了 GROUP BY 年, 大区 的方体，不仅可以回答“2023年华东区销售额”，还可以通过一次简单查询计算出“2023年全球总销售额”（只需对这个方体做一次聚合）。
维护开销 (Maintenance Cost)	考虑当底层数据更新时，更新物化视图的代价。	选择物化那些在数据更新时更容易进行增量刷新（而非全量重算）的方体。

数据方计算

- 用DMQL定义和计算数据方

define cube sales[item, city, year]: sum(sales_in_dollars)

compute cube sales

- 将它变换成类——SQL语句 (用新的操作 **cube by**扩充, 由Gray 等'96 引进)

SELECT item, city, year, SUM (amount)

FROM SALES

CUBE BY item, city, year

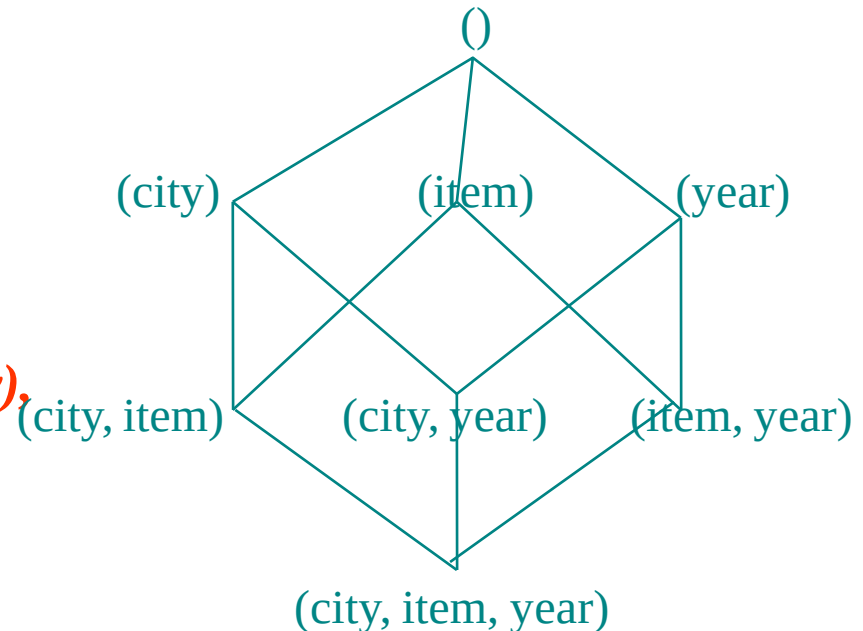
- 需要计算的分组

(city, item, year),

(city,item),(city, year), (item, city),

(city), (item), (year)

()



索引|OLAP 数据

- 为了有效的访问，大部分数据仓库系统支持索引结构
- 两种常用的方法对OLAP数据进行索引
 - 位图索引 bitmap indexing
 - 连接索引 join indexing

索引|OLAP 数据: 位图索引

- 在一个特定列上索引
- 列上的每个值是一个位向量：位操作很快
- 位向量的长度: 基本表的记录数
- 如果数据表中给定行的属性值为 v , 则在位图索引的对应行, 表示该值的位为1, 该行的其它位均为0
- 不适合势(不同值个数)很高的域

基本表

Cust	Region	Type
C1	Asia	Retail
C2	Europe	Dealer
C3	Asia	Dealer
C4	America	Retail
C5	Europe	Dealer

在 Region上索引

RecID	Asia	Europe	America
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1
5	0	1	0

在 Type上索引

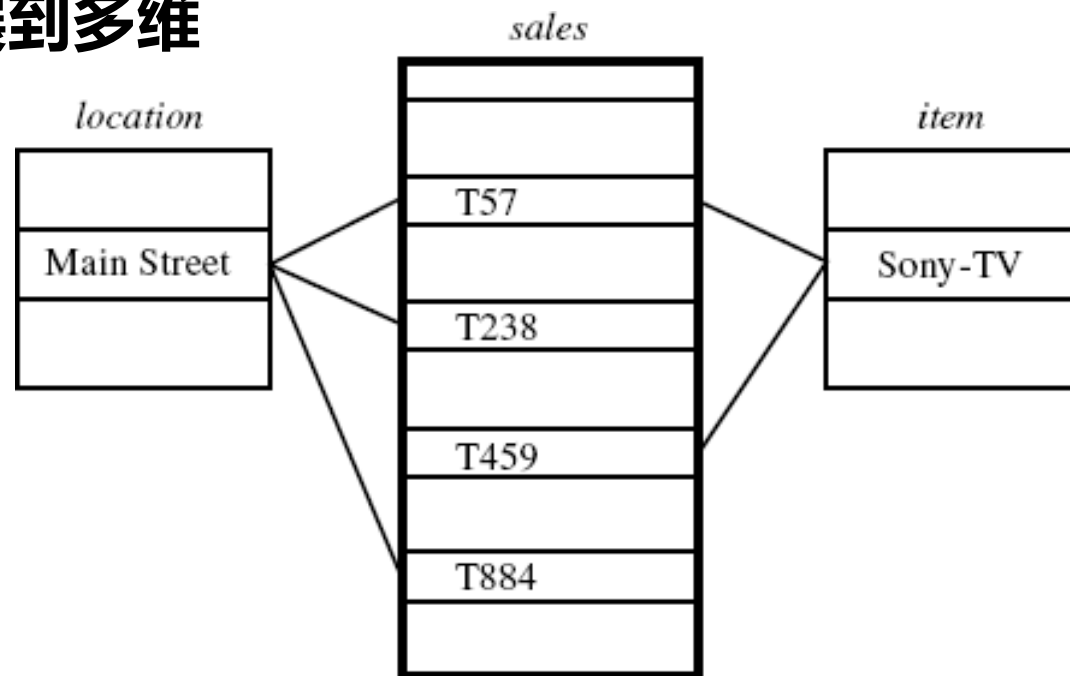
RecID	Retail	Dealer
1	1	0
2	0	1
3	0	1
4	1	0
5	0	1

索引|OLAP 数据: 连接索引

- 连接索引: **JI**(R-id, S-id) , 其中 $R(R\text{-id}, \dots) \triangleright \triangleleft S(S\text{-id}, \dots)$
 - 将关系的连接物化在**JI**文件中, 加快了关系连接的速度
- 数据仓库中, 连接索引将星型模式维表的值关联到事实表的行.
 - 例, 事实表 *Sales* 和两个维 *city* 和 *product*
 - *city* 上的连接索引对每个不同的城市, 维护一张记录该城市销售的元组的R
 - 连接索引可以扩展到多维

Join index table for
item/sales

<i>item</i>	<i>sales_key</i>
...	...
Sony-TV	T57
Sony-TV	T459
...	...



OLAP查询的有效处理

- 物化方体和构造OLAP索引结构的目的是加快数据立方体的查询处理速度。
- 查询处理按如下步骤进行：
- 确定哪些操作可以在可用的方体上进行：
 - 将下钻, 上卷等操作变换成对应的SQL和/或OLAP操作, 例如, $\text{dice} = \text{selection} + \text{projection}$
- 确定相关的操作应当使用哪些物化的方体。

第3章: 数据挖掘的数据仓库与OLAP技术

- 什么是数据仓库?
- 多维数据模型
- 数据仓库结构
- 数据仓库实现
- 从数据仓库到数据挖掘
- 数据立方体的进一步发展

数据仓库使用

- 数据仓库应用的三种类型

- 信息处理

- 支持查询, 基本统计分析, 使用交叉表, 表, 图表和图进行报告

- 分析处理

- 数据仓库数据的多维分析
 - 支持基本的 OLAP 操作, 切片-切块, 上下钻, 转轴

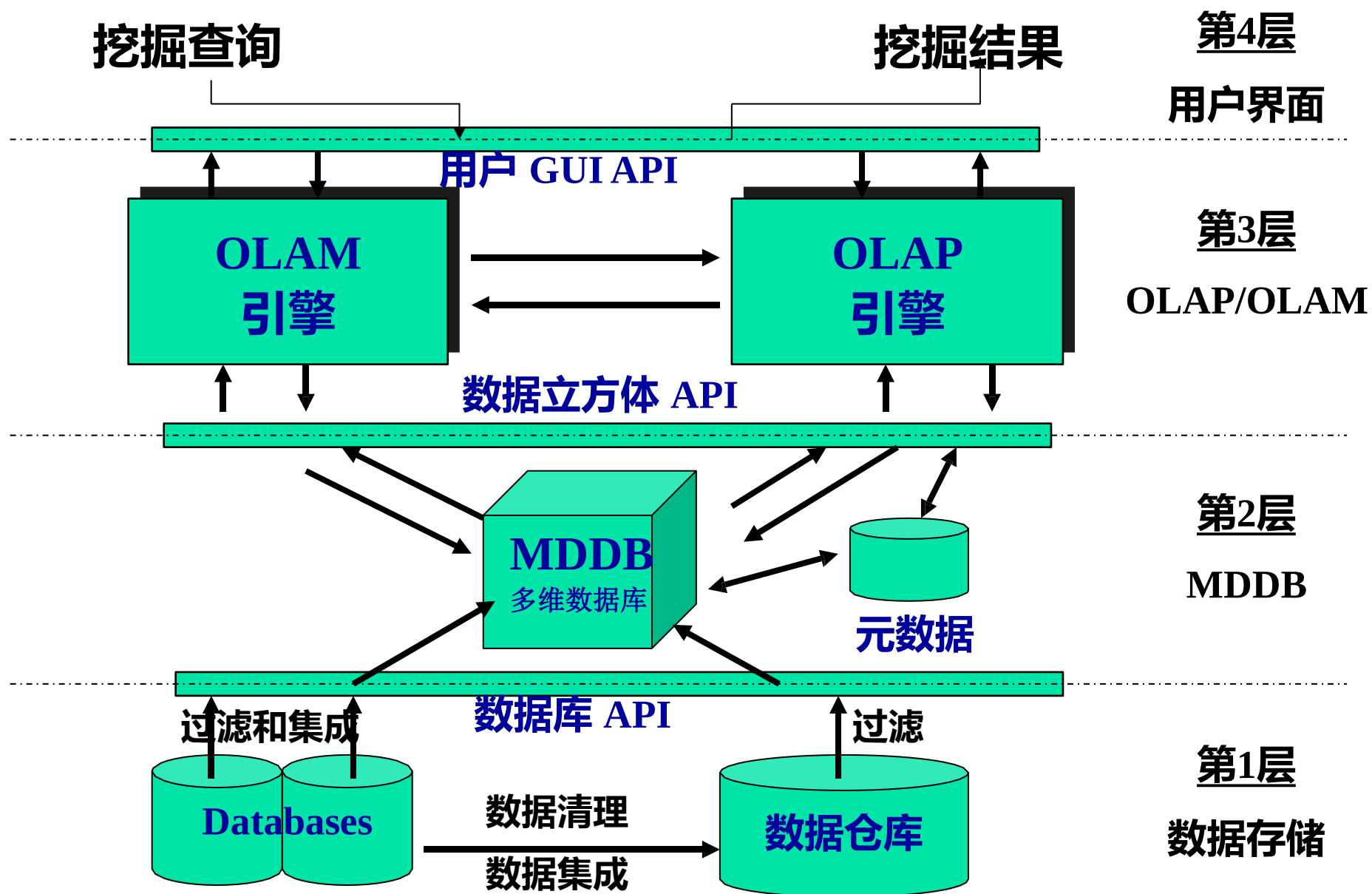
- 数据挖掘

- 隐藏模式的知识发现
 - 支持关联, 构造分析模型, 进行分类和预测, 并使用可视化工具提供挖掘结果.

从联机分析处理到联机分析挖掘

- 为什么要进行联机分析挖掘(OLAM)?
 - 数据仓库中数据的高质量
 - 数据仓库包含集成的, 一致的, 清理过的数据
 - 围绕数据仓库的有价值的信息处理基础设施
 - ODBC, OLEDB, Web 访问, 服务机制, 报告 和 OLAP 工具
 - 基于OLAP的探测式数据分析
 - 使用上下钻, 切片, 切块, 转轴等进行挖掘.
 - 数据挖掘功能的联机选择
 - 集成多种挖掘功能, 算法和任务, 并进行切换.
- OLAM的结构

OLAM 的结构



小结

- **数据仓库**
- **数据仓库的 多维数据模型**
 - 星型模式, 雪花模式, 事实星座
 - 数据方由维和度量组成
- **OLAP 操作: 下钻, 上卷, 切片, 切块 和转轴**
- **OLAP 服务器: ROLAP, MOLAP, HOLAP**
- **数据方的有效计算**
 - 部分 vs. 全部 vs. 不物化
 - 多路数组聚集
 - 位图索引和连接索引的实现

